Minh & Nicklas og Adem
– LagerKlar app –

1. **Problemstilling**.

I mindre restauranter kan det være svært at holde styr på hvilke varer der er på lager, og hvornår de udløber. Nogle gange bliver lageret skrevet ned på papir eller husket fra dag til dag. Så kan det være, at man opdager for sent at en vigtig vare mangler. Det kan også ske at varer bliver glemt og udløber før de bliver brugt. Det giver både ekstra udgifter og madspild.
Vi vil derfor undersøge hvordan små restauranter lettere kan holde overblik over deres varer og lager. Vores problemformulering er:

Hvordan kan vi hjælpe små restauranter med at holde bedre styr på deres lager og udløbsdatoer, så de kan undgå at mangle varer og mindske madspild?

2. **Målgruppe**.

- Små restauranter, cafeer og lignende spisesteder
- Medarbejdere der registrerer varer og opdaterer lagerbeholdning
- Leder eller ansvarlig for indkøb der skal have overblik og påmindelser

Formålet er at samle lagerregistrering og udløbsoverblik i en mobil løsning, som kan bruges under det daglige arbejde.

3. Ressourcer og antagelser.

- Projektet planlægges til tre personer i fire uger. Der er behov for udvikling computere, GitHub, test på iOS og Android, et servermiljø, en database og eventuelt billedlager. Adgang til iOS-testmiljø, mobile test og eventuelle driftsudgifter afklares i første etape.
- Vi antager, at brugerne har internetadgang og tilladelse til at bruge kameraet, at restauranten registrerer lagerændringer løbende, og at udløbsdatoer indtastes manuelt ved oprettelse. En QR-kode identificerer et vareparti; den henter ikke automatisk en udløbsdato fra internettet.

4. Kravspecifikation og MoSCoW.

- **Funktionelle krav**
  - Brugeren skal kunne logge ind med en brugerkonto
  - Brugeren skal kunne oprette en ny vare med navn, kategori, mængde, enhed, indkøbsdato og udløbsdato.
  - Brugeren skal kunne slette en vare fra lageret.
  - Brugeren skal kunne søge efter en vare ud fra navn eller kategori
  - Brugeren skal kunne indstille hvor mange dage før udløb en påmindelse skal sendes
  - Flere bruger skal kunne tilgå samme lager

    - Systemet skal kunne vise en oversigt over alle varer på lageret
    - Systemet skal kunne opdatere lagerbeholdningen når varer bruges, modtages eller kasseres
    - Systemet skal kunne vise varer der snart udløber i en egen oversigt
    - Systemet skal kunne vise varer med lav lagerbeholdning i en egen oversigt
    - App kan generere stregkode eller qr-code til brugte varer.
- **Ikke-funktionelle krav**
    - App'en skal være nem at bruge for medarbejdere uden teknisk erfaring
    - Data om varer skal gemmes sikkert og kun være tilgængelige for autoriserede brugere
    - Systemet skal være tilgængeligt når restauranten er åben

Afgrænsning af, hvad produktet skal kunne (MoSCoW).

Must have(MVP):

- Login og register med **email**.
- Appen kan bruges i både iOS og Android.
- Database bruges til at gemme brugervirksomheder.
- Brugersystem.
- Image storage.
- Visse varer, der snart udløber, og sende påmindelser..

Should have:

- Eksport af lageroversigt eller indkøbsliste til CSV.
- Kryptering på DB.
- Appen skal sikre at brugerne skal kun blive login en af gange.
- Alarmsystemet for daglig tjekke.
- Leder roller kan selv tilføje ny medarbejder i deres virksomhed.
- Lederrollen skal kunne se oversigt over varer der snart udløber
- Lederrollen skal kunne se oversigt over varer med lav lagerbeholdning
- Lederrollen skal kunne deaktivere en medarbejder konto når medarbejderen ikke længere er ansat
- Lederrollen skal kunne oprette og redigere minimumsgrænser for vare

Could have:

- Brugertyper som leder (leder / medarbejder mm..)
- Central Image Storage via Firebase.
- Flere restauranter eller afdelinger i samme løsning
- Chat function
- Lederrollen skal kunne se hvilke medarbejdere der har opgraderet lageret
- Lederrollen skal kunne se en indkøbsliste baseret på varer under deres minimumsgrænse

- Brugerroller:
    - **Medarbejder**
      Kan se lageret, registrere brugte varer, opdatere mængder og foretage lageroptælling
    - **Leder / indkøbsansvarlig**
      Kan oprette og redigere varer, ændre minimumsgrænser, se rapporter og modtage påmindelser

Will not have:

- Kompliceret UI med mange niveauer
- Avanceret dashboard
- Customizable hjemmeskærm eller wigets
- Integration med kasseapparater, POS-systemer, leverandør systemer eller regnskabsprogrammer
- Takeaway og salgsfunktioner med betaling eller ordrebehandling.

5. Use Case UC-03 – Registrere varer via QR-kode

Primær aktør: Medarbejder

Formål: Medarbejderen skal kunne opdatere lageret, når varer bliver solgt, brugt eller kasseret.

Start: Medarbejderen scanner QR-koden på et vareparti.

Forudsætninger:

- Medarbejderen er logget ind.
- QR-koden er knyttet til et vareparti i systemet.

Hovedforløb

1. Medarbejderen scanner QR-koden på partiet.
2. Systemet finder partiet og viser varenavn, udløbsdato og aktuel beholdning.
3. Medarbejderen vælger, om varen er solgt, brugt eller kasseret.
4. Medarbejderen indtaster det antal, der skal trækkes fra lageret.
5. Systemet kontrollerer, at antallet ikke er større end den aktuelle beholdning.
6. Medarbejderen bekræfter ændringen.
7. Systemet gemmer lagerændringen og viser den nye beholdning.

Alternativt forløb

- Hvis QR-koden ikke kan findes, viser systemet en fejlbesked.
- Hvis medarbejderen indtaster et for stort antal, beder systemet om et nyt antal.

- Hvis medarbejderen annullerer, bliver lageret ikke ændret.

Resultat: Lagerbeholdningen er opdateret, og ændringen er gemt med tidspunkt og type.

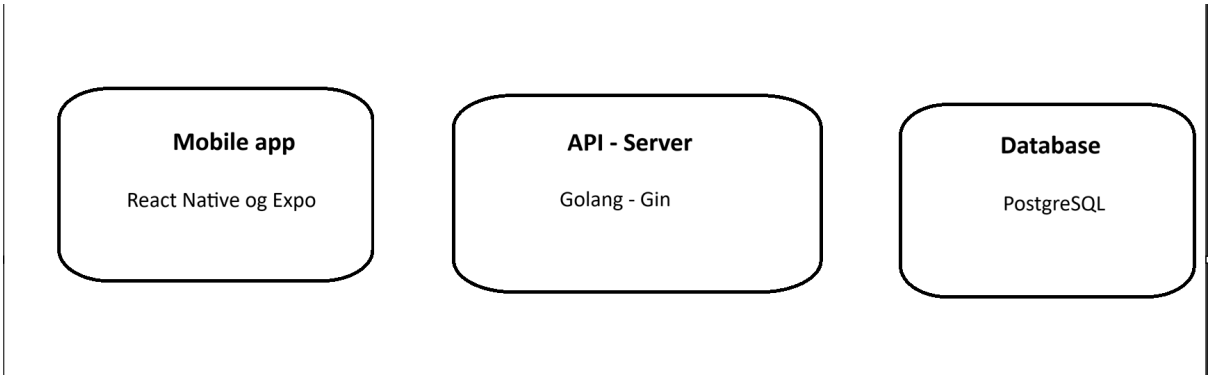
Brugerforløb fra registrering til forbrug

Case	Brugerens handling	Systemets reaktion
1.0 – Registrering	Vælger »Opret konto«.	Viser registreringsformularen.
1.1	Indtaster navn, e-mail og adgangskode.	Kontrollerer oplysningerne.
1.2	Trykker på »Opret konto«.	Opretter kontoen og viser login-skærmen.
1.2a	Indtaster en e-mail, der allerede er registreret.	Viser en fejlbesked og henviser til login.
2.0 – Login	Indtaster e-mail og adgangskode.	Kontrollerer loginoplysningerne.
2.1	Trykker på »Log ind«.	Logger brugeren ind og viser restaurantens lager.
2.1a	Indtaster forkerte loginoplysninger.	Viser en fejlbesked og giver mulighed for at prøve igen.
3.0 – Se lager	Åbner lageroversigten.	Viser restaurantens varer og beholdninger.
3.1	Søger efter navn eller kategori.	Viser de varer, der matcher søgningen.
4.0 – Opret vareparti	Vælger »Tilføj vare«.	Viser formularen til oprettelse.
4.1	Indtaster varenavn, kategori, mængde, enhed, indkøbsdato og udløbsdato.	Kontrollerer oplysningerne.
4.2	Trykker på »Gem«.	Gemmer varen og varepartiet samt genererer en QR-kode til partiet.
4.2a	Mangler obligatoriske oplysninger.	Markerer felterne og beder brugeren udfylde dem.
5.0 – Registrer forbrug	Vælger »Scan QR-kode« og scanner partiet.	Viser varenavn, udløbsdato og aktuel beholdning.
5.1	Vælger solgt, brugt eller kasseret og indtaster antal.	Viser ændringen til bekræftelse.

5.2	Bekræfter ændringen.	Kontrollerer antallet, opdaterer beholdningen og gemmer tidspunkt, type og medarbejder.
5.2a	Indtaster et antal på nul, et negativt antal eller mere end beholdningen.	Afviser ændringen og beder om et gyldigt antal.
5.2b	Annullerer registreringen.	Afslutter uden at ændre lageret

6. Foreløbig løsning.

Overordnet systemskitse.



Redegørelse for valg af tech stack.

- Vi beslutter at bruge **React native** med **Expo** for at den kan køre i både iOS og Android. Vi har også arbejdet i den framework før, så det er nemt at sætte i gang hurtig.
- **Expo Dev Client**: Vi har valgt Expo Dev Client, fordi det giver mulighed for at bruge native funktioner og konfigurere tilladelser ud over det, Expo Go understøtter, eksempelvis til push notifikationer. Kamera Adgang understøttes også i Expo Go.

- **Database – PostgreSQL**: Vi har valgt PostgreSQL, De relationelle forbindelser mellem virksomhed, brugere, varer, partier og lagerændringer gør en relationel database relevant. PostgreSQL understøtter SQL og kan også gemme fleksible data i JSON-format.
- **Backend - Golang**:

7. Tidsplan.

Uge	Dato	Opgaver	Ansvarlig	Estimate
1		1. Databasestruktur design. 2. Oprette et projekt i Github.	1.Minh 2. Nicklas	1. 1,5 dag 2. 1 dag
1		1. Oprette Go-projekt 2. Oprette et React Native-projekt med javascript. 3. Docker setup - databasemigrationer	1. Adem 2. Nicklas 3.Minh	1. 1 dag 2. 1 dag 3. 1 dag
1		1. Lave skærme til registrering og login i appen. 2. Lave backend til login og registrering. 3. Klargøre databasen til at gemme brugeroplysninger.	1.Nicklas 2.Adem 3.Minh	1. 2 dage 2. 2 dage 3. 1 dag
1		1. Lave UI til lagerskærmen. 2. Klargøre backend-endpoints til lagerskærmens funktioner. 3. Implementere autentificering og hashing af adgangskoder i backend.	1.Nicklas 2.Adem 3. Minh	1. 1 dag 2. 2 dage 3. 2 dage
1		1. Gennemgå processen og rette fejl. 2. Arbejde med roller i appen og definere, hvem der har adgang til hvad.	1. Minh 2. Nicklas	1. 1 dag 2. 3 dage

2		1. Lave oprettelse og endpoints til at oprette, redigere og sletning af varer. 2. Oprette varepartier i databasen. 3. Generate QR-code i app og gemme i database.	1. Nicklas 2.Adem 3.Minh	1. 2 dage 2. 2 dage 3. 1 dag
2		1. Implementere scanning og vise oplysninger i Appen (Frontend + backend). 2. Implementere registrering af modtagne, brugte og kasserede varer. 3. Vise varer, der snart udløber, og varer med lav beholdning i appen.	1.Minh 2.Adem 3. Nicklas	1. 2 dage 2. 2 dage 3. 2-3 dage
2		Implementere og teste første version af påmindelser (Expo token).	Minh	1,5 dage
2		Færdiggøre registrering af modtagne, brugte og kasserede varer. Kontrollere antal og gemme ændringer.		
2		Teste varer, scanning og lagerændringer. Rette fejl og holde ugereview.	Fælle	
3		Vise varer med kort holdbarhed og lav beholdning. Lave minimumsgrænser.		
3		Lave indstillinger for påmindelser og daglig kontrol af udløbsdatoer.	Minh	1,5 dage
3		Sende og teste påmindelser på iOS og Android.	Minh	1 dag
3		Lade lederen tilføje og deaktivere		

		medarbejdere.		
3		Lave upload og visning af varebilleder og holde ugereview.		
4		Teste hele appen ud fra use cases på iOS og Android. Gennemføre brugertest.		
4		Rette fejl og forbedre brugervenligheden.		
4		Færdiggøre dokumentation og testresultater.		
4		Afprøve relevante rettelser med brugere. Genteste kritiske fejl. Klargøre installation og demo.		
4		Afsluttende review, demonstration og aflevering.		